

Chapter 34

Key Manager

34.1 Overview

This Key Management module is the security core of the system. It has a Key Manager and a Hardware Unique Key (HUK) Generator. They can store and deploy keys securely. By utilizing the HUK as the root of trust for each chip, the system ensures that no plaintext keys are ever stored on the chip, enhancing its overall security.

Compared with using eFuse to store keys, the Key Manager and HUK Generator can improve the system in the following ways:

- Environmental adaptability: It supports secure key deployment and can complete key deployment safely, even if the external environment is untrusted.
- Anti-attack ability: Each chip's SRAM has a unique default value that software cannot access. The chip stores no plaintext key. It resists slice and copy attacks. For example, observing the keys in eFuse directly by slicing will fail.
- Key management flexibility: The Key Manager enables users to store key information (non-plaintext) in external memory. Unlike eFuse, which has limitations on the number of keys it can store, Key Manager allows users to create an unlimited number of keys, perform dynamic key changes, key replacements, and etc.

34.2 Background Knowledge

An SRAM PUF (Physical Unclonable Function) uses the natural power-up state of SRAM cells as a kind of silicon “fingerprint.” When a chip powers on, each uninitialized SRAM bit tends to fall into 0 or 1 based on tiny manufacturing differences in its transistors, and because those differences are unique for every chip, the resulting bit pattern is device-unique.

We use this behavior to create a Hardware Unique Key (HUK) without ever storing that key anywhere on chip: the chip can regenerate its secret from physics on each boot instead of reading it from flash or fuses. The HUK then acts as the root of trust for secure boot, device identity and attestation, encryption keys, and wrapping other keys.

The raw SRAM PUF bits are not perfectly stable, though: most bits come up the same way on every power cycle, but some are noisy and occasionally flip due to temperature, voltage, or noise. In order to generate a stable HUK, the chip generates auxiliary data (huk_info) with error correction capabilities during the HUK generation stage to recover the HUK. Users need to store this auxiliary data in flash memory.

On every later boot, hardware reads the new (noisy) PUF response, uses the helper data to correct any flipped bits back to the reference pattern, and reconstructs exactly the same HUK as during enrollment. That HUK is

loaded into hidden key slots inside crypto hardware (never into normal RAM), giving the device a repeatable, per-chip unique secret key derived from SRAM PUF, even though the underlying bits are not 100% identical at every power-up.

34.3 Glossary

To better illustrate the functions of the HUK Generator and Key Manager, the following terms and parameters are used in this chapter.

Terms

HUK	Hardware Unique Key (HUK), a hardware-generated key derived from the default power-on value using SRAM PUF.
SRAM PUF	Physical Unclonable Function (PUF), based on SRAM, a security technology that leverages the unique, unpredictable power-up behavior of SRAM to generate a unique fingerprint or cryptographic key for a device.
HUK Generation Mode	In this mode, the HUK Generator creates a new HUK.
HUK Recovery Mode	In this mode, the HUK Generator recovers a previously deployed HUK.
Random Deploy Mode	In this mode, the Key Manager deploys a random private key generated by the Random Number Generator.
AES Deploy Mode	In this mode, the Key Manager deploys a specified private key ($k1$) onto the chip.
ECDH0 Deploy Mode	In this mode, the Key Manager deploys a negotiated private key ($k1 * k2 * G$) onto the chip. $k1 * k2 * G$ is generated using $k1$ and $k2 * G$.
ECDH1 Deploy Mode	In this mode, the Key Manager deploys a negotiated private key ($k1 * k2 * G$) onto the chip. $k1 * k2 * G$ is generated using $k1$, $k2 * G$, and $k2_info$.
Private Key Recovery Mode	In this mode, the Key Manager recovers a previously deployed private key.
<i>key_info</i> Export Mode	In this mode, the Key Manager regenerates the <i>key_info</i> . Users can export this <i>key_info</i> from the internal memory of the Key Manager.

Parameters

<i>huk_info</i>	Contains fuzzy extraction information and error-correcting code information after RS coding of HUK, but does not contain any information of HUK itself. This <i>huk_info</i> later can be used to recover the deployed HUK.
<i>key_info</i>	In any working mode of the Key Manager, this <i>key_info</i> is generated by calculating $AES\{AES\{key, rand_num\} rand_num, HUK\}$.
$AES\{a, b\}$	Encrypt plaintext a with b as the key, using AES_ECB algorithm, which is the typical mode of the AES module.
<i>rand_num</i>	A random number
$a b$	Concatenates two pieces of data into one.
$k1$	A specified private key

<i>k2</i>	Auxiliary key used to assist with key deployment, applicable in both AES Deploy Mode and ECDH1 Deploy Mode. In AES Deploy Mode, its value does not affect the deployed key. In ECDH1 deployment mode, its value affects the negotiated deployment key.
<i>k1_encrypted</i>	A key generated by encrypting <i>k1</i> with <i>k2</i> . In AES Deploy Mode or ECDH1 Deploy Mode: $k1_encrypted = AES\{k1, k2\}$
<i>k2_info</i>	The recovery information of <i>k2</i> . In AES Deploy Mode or ECDH1 Deploy Mode: $k2_info = AES\{AES\{k2, rand_num\} rand_num, init_key\}$.
<i>init_key</i>	Used in the generation of <i>k2_info</i> as well as in the recovery process from <i>k2_info</i> to <i>k2</i> . Its value can be stored in the eFuse block whose purpose is EFUSE_KM_INIT_KEY or provided through software configuration.
<i>sw_init_key</i>	Software configured <i>init_key</i>
$k1 * k2 * G$	A negotiated key generated by calculating $k2 * G$ through ECC.
$k1 * G$	A key generated by calculating <i>k1</i> through ECC.
$k2 * G$	An auxiliary public key generated by calculating $k2 * G$ through ECC. This key is used later to generate $k1 * k2 * G$.
<i>key_purpose</i>	The intended usage of the key.

34.4 Features

The HUK Generator has the following features:

- HUK Generation Mode:
 - Generates a new HUK and its recovery information
- HUK Recovery Mode:
 - Recovers a deployed HUK with its recovery information
- Prompt for HUK recovery error
- Prompt for HUK risk level

The Key Manager has the following features:

- Unlimited number of keys
- Specified private key deployment (AES Deploy Mode):
 - Users specify the value of the key
- Negotiated private key deployment (ECDH0 Deploy Mode):
 - Highest security mode: there is no need to worry about the leaks of data in external channels
 - Requires chip initiation to obtain the private key
 - Negotiates the key value between each chip and the user
- Negotiated private key deployment (ECDH1 Deploy Mode):

- Deploy the negotiated private key using the auxiliary key
- No need to boot the chip to obtain the private key
- Random key deployment (Random Deploy Mode):
 - Deploys a hardware-generated random key with nobody knowing the exact value
- Private key recovery deployment (Private Key Recovery Mode):
 - Recovers exactly the same key by entering the key information generated during deployment
- Key information export (*key_info* Export Mode):
 - Generates unique key information for the same key each time

34.5 Architectural Overview

The top-level architecture of the Key Manager and HUK Generator is shown in the figure below.

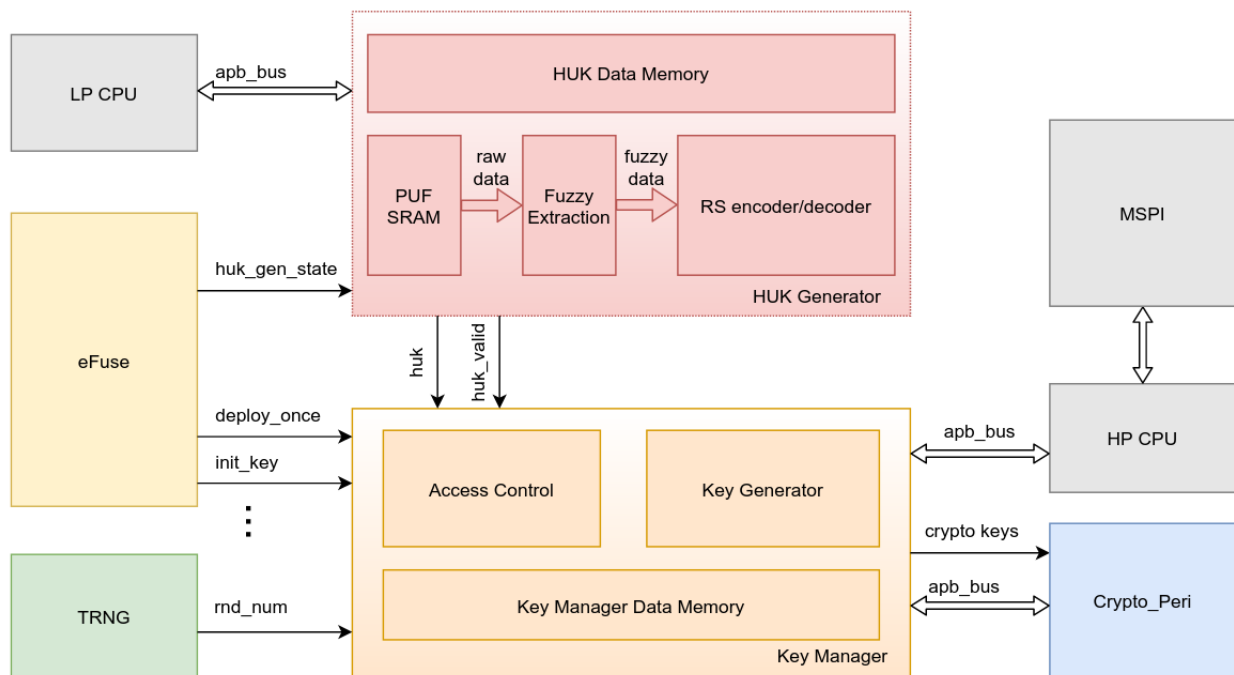


Figure 34.5-1. Architecture of Key Manager and HUK Generator

The Key Manager has the following interaction paths with external modules:

- **eFuse path:** Controls the configuration of Key Manager with the help of eFuse bits.
- **HP APB path:** Configures registers and transmits memory data via HP APB bus.
- **HUK path:** Receives HUK from HUK Generator.
- **Crypto path:** Controls cryptographic accelerators such as ECC/AES to complete cryptographic operations via Crypto APB bus.
- **TRNG path:** Receives true random data from the TRNG.

The HUK Generator has the following interaction paths with external modules.

- **eFuse path:** Controls the configuration of HUK Generator with the help of eFuse bits.
- **LP APB path:** Configures registers and transmits memory data via LP APB bus.
- **Key Manager path:** Sends HUK to Key Manager.

34.6 Functional Description

34.6.1 HUK Generator

HUK Generator generates unique HUKs for each chip based on SRAM Physical Unclonable Function (PUF). This HUK is completely inaccessible by software, ensuring robust security for the Key Manager, which relies on this HUK for secure operations later on.

34.6.1.1 HUK Generation Process

The HUK Generator has the following modes:

- **HUK Generation Mode:** Creates a new HUK and generates corresponding HUK recovery information (*huk_info*).
- **HUK Recovery Mode:** Recovers HUK according to *huk_info*.

The modes above are jointly controlled by the eFuse field EFUSE_KM_HUK_GEN_STATE, and the register HUK_MODE, as shown in the following table.

Table 34.6-1. HUK Generator Working Mode

HUK_MODE	eFuse *	Mode
1	Total number of bits set to 1 is even	HUK Generation Mode
	Total number of bits set to 1 is odd	HUK Recovery Mode
0	—	HUK Recovery Mode

* Total number of bits set to 1 in EFUSE_KM_HUK_GEN_STATE.

Note:

- Once the process is completed in HUK Generation Mode, it is recommended to write a 1 to EFUSE_KM_HUK_GEN_STATE field, ensuring that the total number of bits set to 1 in EFUSE_KM_HUK_GEN_STATE is an odd number. With this, when the chip starts next time, the HUK Generator can only enter HUK Recovery Mode. The HUK restored in HUK Recovery Mode will be the same as the one generated this time in HUK Generation Mode.
- If you do not update the eFuse bits EFUSE_KM_HUK_GEN_STATE field after completing the HUK Generation Mode and before powering off the chip, the HUK Generator can still be configured to enter HUK Generation Mode again when the chip is powered on next time. The newly generated HUK this time will have a different value from the previous one that is used in private keys. By this way, you can generate multiple HUKs and restore the one you need with its *huk_info*.
- *huk_info* can only be used with the same chip; that is, an *huk_info* generated on chip A can not be used to recover the HUK on chip B.

34.6.1.2 HUK Status Check

Users can read the register [HUK_STATUS_REG](#) to check the current status of the HUK and decide whether to regenerate the HUK. [HUK_STATUS_REG](#) has the following fields:

- [HUK_STATUS](#): Indicates the current status of the HUK.
 - 0: The HUK has not been generated
 - 1: The HUK has been successfully generated and is valid
 - 2: The HUK has been generated but is invalid
 - 3: Reserved
- [HUK_RISK_LEVEL](#): Indicates the current risk level of the HUK. The value ranges from 0 to 7, with higher values indicating greater risk. When the value is 7, the HUK is no longer available. At this point, the [HUK_STATUS](#) will be set to 2.

Note:

- HUK Generator uses the default value of SRAM PUF to generate HUK. The HUK risk level refers to the difference between the current default value of SRAM PUF and the default value of SRAM PUF at the time when the HUK was created.
- In certain special applications, such as when users want to discard a key and invalidate the corresponding [key_info](#), they can select to update the HUK. Once the HUK is changed, the previously deployed key, as well as its [key_info](#), both become invalid, i.e., the new [huk_info](#) together with the previous [key_info](#) can not be used to recover the key.
- It is strongly recommended that software monitor the risk level and regenerate the HUK at the value 4-6.

34.6.2 Key Manager

Note:

Each time before using the Key Manager for key deployment, please ensure that the HUK is valid by checking the register [HUK_STATUS_REG](#).

Key Manager provides the following modes:

- **Key Deployment Modes:**
 - Random Deploy Mode: Deploys a random private key.
 - AES Deploy Mode: Deploys a specified private key.
 - ECDH0 Deploy Mode: Deploys a negotiated private key.
 - ECDH1 Deploy Mode: Deploys a negotiated private key.

These four deployment modes allow users to select the most convenient deployment process in environments with different security levels according to their own needs.

- **Key Recovery Mode:**
 - Private Key Recovery Mode: Recovers the deployed private key.

This recovery mode allows users to freely select the key they want to recover from any number of the deployed keys.

- **Export Mode:**

- *key_info* Export Mode: Generates and outputs *key_info* of the deployed private key.

This export mode allows users to regenerate *key_info* for already deployed keys.

34.6.2.1 Random Deploy Mode

Users can use this mode to deploy a random key generated with the help of the Random Number Generator.

Note:

The Random Deploy Mode is highly convenient and well-suited for application scenarios where the exact value of the private key does not need to be known.

34.6.2.2 AES Deploy Mode

Users can use this mode to deploy a specified private key.

This mode requires the following keys and parameters:

- HUK: generated by the HUK Generator and passed to the Key Manager.
- *k1*: the specified private key to be deployed.
- *k2*: Auxiliary key used to encrypt *k1*, ensuring that *k1* is not transmitted in plaintext over the communication link, thereby enhancing data security.
- *init_key*: Initial auxiliary key used for generating and decrypting the *k2_info* key.
- *k2_info*: Generated from *k2* and *init_key*, and used to recover *k2*. The calculation formula is:
$$k2_info = AES\{AES\{k2, rand_num\} || rand_num, init_key\}.$$

The deployment flow chart is as follows.

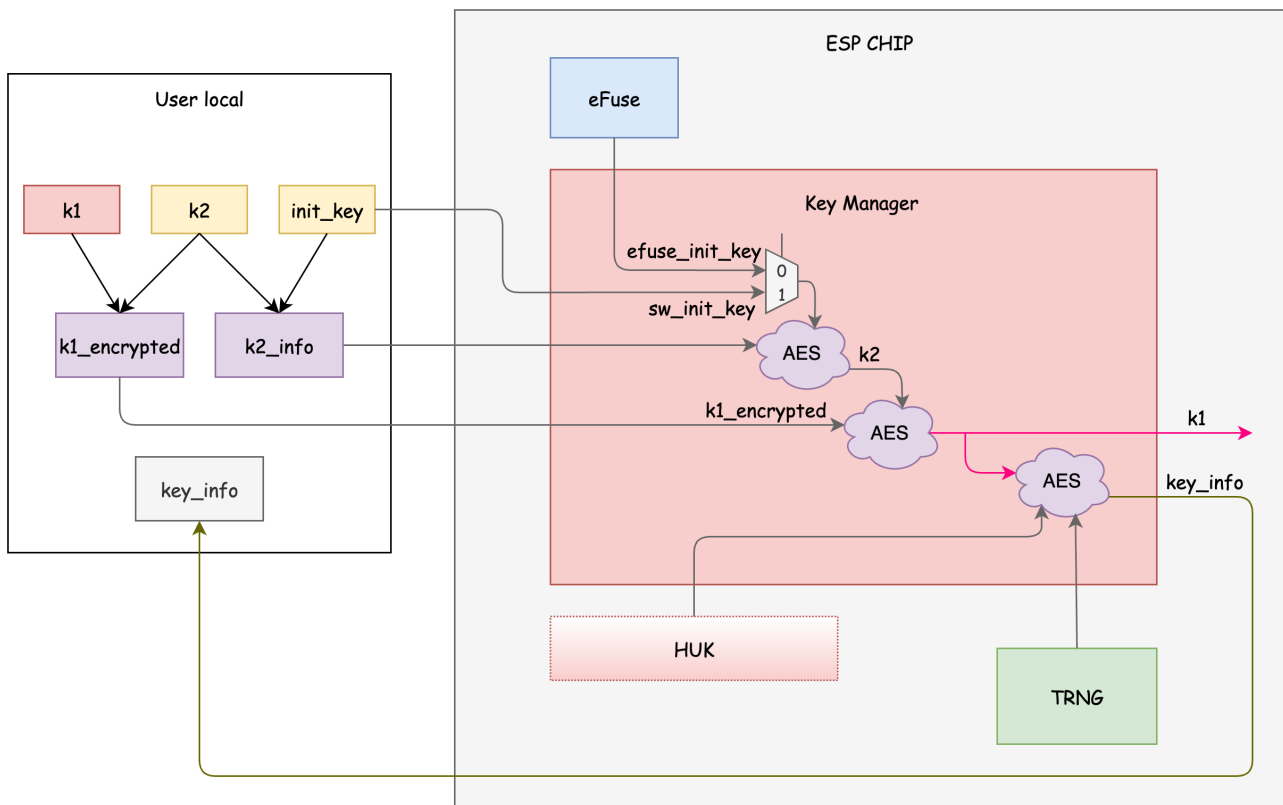


Figure 34.6-1. AES Deploy Mode

Detailed Process

1. Generate locally *k1*, *k2*, and *init_key*.
2. Calculate locally *k1_encrypted* and *k2_info*.
3. Then complete the key deployment in Key Manager:
 - (a) Write *init_key* into the eFuse block whose purpose is EFUSE_KM_INIT_KEY, or to *sw_init_key*, depending on your need.
 - (b) Write *k2_info* and *k1_encrypted* into internal memory of the Key Manager.
 - (c) Wait for the Key Manager to complete the deployment.
 - (d) Read *key_info* from the internal memory of the Key Manager by specifying the address.

Notes:

- Due to the necessity of re-deploying the key each time when the chip powers up, if *key_info* is to be used for key recovery upon the next power-up, ensure that *key_info* must persist after the chip powers off. A common method is to store it in the flash.
- AES Deploy Mode is suitable for factory production: the manufacturer provides the factory with *init_key* and the firmware encrypted with *k1*, and store *k2_info* and *k1_encrypted* in the firmware.
 - In this case, it is recommended to include a private key first-time deployment process in the firmware, to store *key_info*. In the subsequent chip power-ups, use the recovery deployment mode, instead of deploying via AES mode each time. This helps effectively defend against eFuse slice attacks.

- Note that this deployment method requires a trusted factory, as the factory will have access to the plaintext *init_key* (used to be burned into eFuse key block). If the factory is untrusted and knows how to use *init_key* to decrypt *k2_info* into *k2*, it may result in a potential leakage of *k1*.
- When using AES Deploy Mode to deploy the key each time, it is essential to ensure that the values of *init_key*, *k2_info*, and *k1_encrypted* are not leaked simultaneously.
 - If using *init_key* stored in the EFUSE_KM_INIT_KEY eFuse block, *k2_info* and *k1_encrypted* may be transmitted in an untrusted environment. However, this requires to locally retain the *init_key* that was written to EFUSE_KM_INIT_KEY eFuse block at the first time.
 - If using *sw_init_key*, the transmission of *sw_init_key*, *k2_info*, and *k1_encrypted* must occur in a trusted environment.

34.6.2.3 ECDH0 Deploy Mode

Users can use this mode to deploy a negotiated private key.

This mode requires the following keys and parameters:

- HUK, generated by the HUK Generator and passed to the Key Manager.
- *k1*: root key, a private key from users.
- $k1 * G$: obtained by calculating $k1 * G$ using ECC.
- $k2 * G$: a public auxiliary key, generated by calculating $k2 * G$ using ECC.
- $k1 * k2 * G$: the negotiated private key to be deployed, generated by Key Manager using $k1 * G$ and $k2$ or by the user locally using $k1$ and $k2 * G$.
- *key_info*: the recovery information of $k1 * k2 * G$.

Any information disclosure during this deployment, occurring outside the users' local environment or the chip, will not affect the generation of $k1 * k2 * G$ or pose a risk of exposure. The deployment flow chart is as follows.

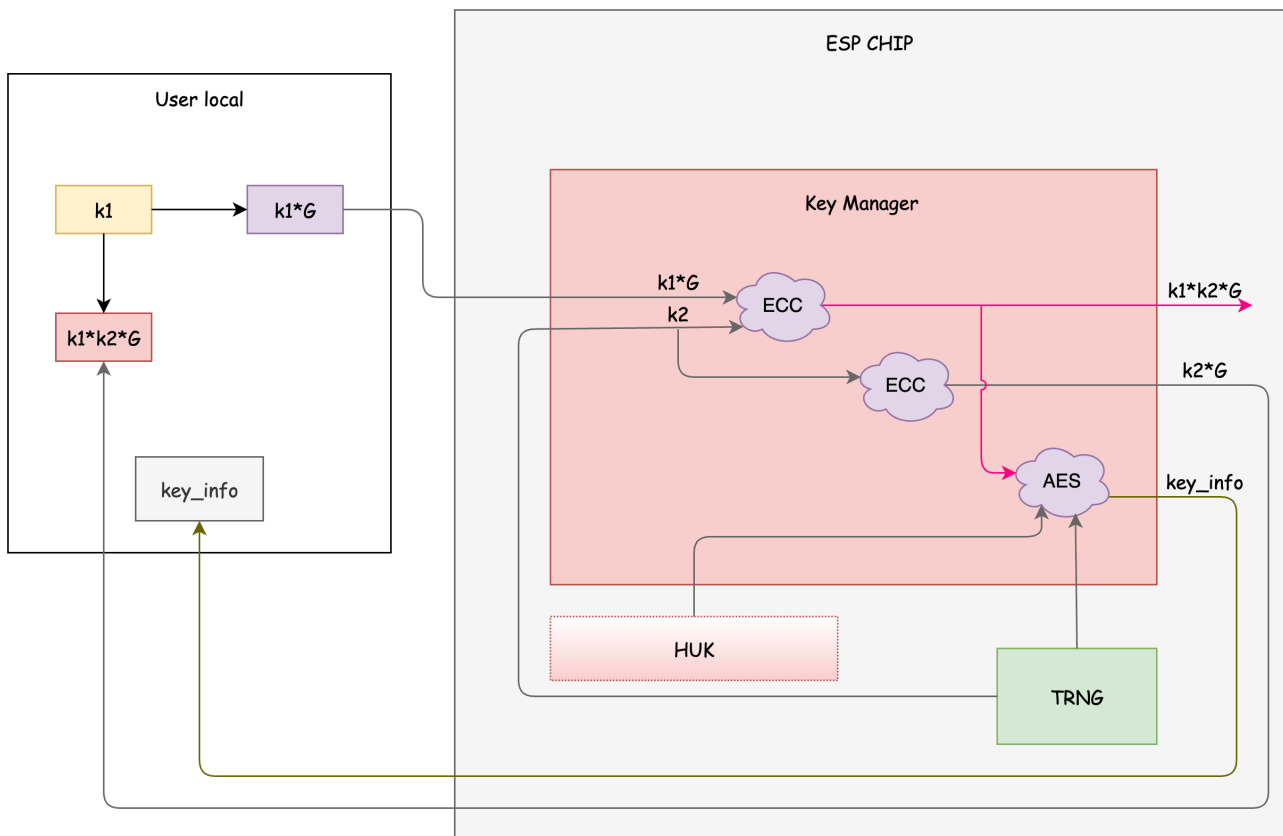


Figure 34.6-2. ECDH0 Deploy Mode

Detailed Process

1. Generate locally k_1 and calculate $k_1 * G$.
2. Complete the key deployment in Key Manager:
 - (a) Write $k_1 * G$ to the internal memory of the Key Manager.
 - (b) Wait for the Key Manager to complete the deployment.
 - (c) Read $k_2 * G$ and key_info (corresponding to the negotiated private key $k_1 * k_2 * G$) from the internal memory of the Key Manager by specifying the address.
3. Generate the negotiated key $k_1 * k_2 * G$ by using $k_2 * G$ together with the local private key k_1 .

Note:

- NIST-P256 curve is used in the ECDH0 mode, all the ECC point multiplication is calculated on NIST-P256 curve.
- The ECDH0 Deploy Mode ensures that the negotiated private key is generated only locally by the user and internally by the Key Manager. Any data leakage during the deployment process will not result in the leakage of the negotiated private key. Therefore, data transmission in this mode can occur in an untrusted environment.
- In ECDH0 Deploy Mode, the negotiated private key can only be obtained after running the Key Manager. This mode is not suitable for scenarios where the exact value of the private key needs to be known in advance.

34.6.2.4 ECDH1 Deploy Mode

Users can use this mode to deploy a negotiated private key.

This mode requires the following keys and parameters:

- HUK, generated by the HUK Generator and passed to the Key Manager.
- $k1$: root key, a private key from users.
- $k1 * G$: obtained by calculating $k1$ using ECC.
- $k2$: an auxiliary private key, in ECDH1 Deploy Mode only existing locally or inside Key Manager.
- $k2 * G$: an auxiliary public key, generated by using the chip supplier's software or script.
- $k2_info$: generated by using $k2$ and $init_key$, and used to recover $k2$. The calculation formula is:
 $k2_info = AES\{AES\{k2, rand_num\} || rand_num, init_key\}$.
- $k1 * k2 * G$: the negotiated private key to be deployed. The chip generates the negotiated private key using $k1 * G$ and $k2_info$, while the user generates the negotiated private key using $k1$ and $k2 * G$.
- key_info : the recovery information of $k1 * k2 * G$.

The deployment flow chart is as follows.

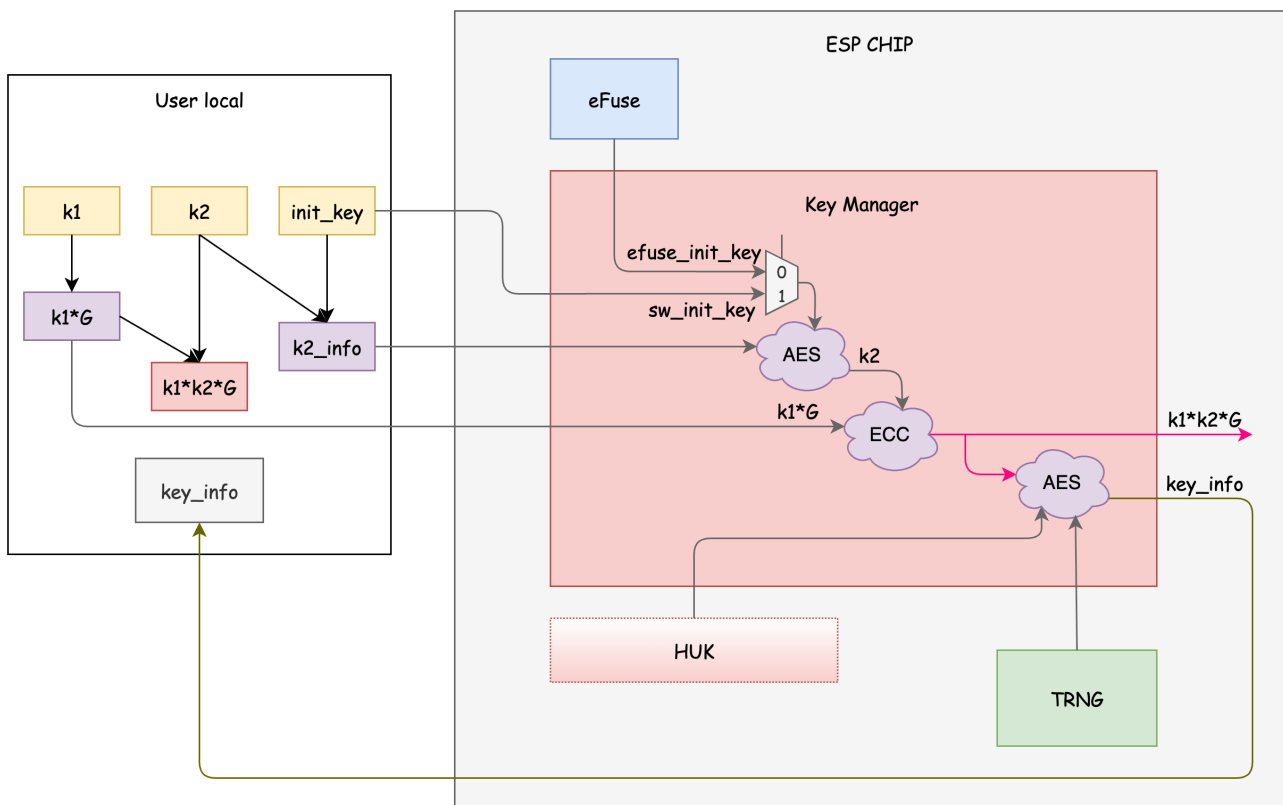


Figure 34.6-3. ECDH1 Deploy Mode

Detailed Process

1. Generate locally $k1$, $k2$ and $init_key$.
2. Calculate locally $k1 * G$, $k2 * G$, $k2_info$, and $k1 * k2 * G$.
3. Complete the key deployment in Key Manager:
 - (a) Write $init_key$ into EFUSE_KM_INIT_KEY eFuse block or into sw_init_key , depending on your needs. See the notes below.

- (b) Write $k1 * G$ and $k2_info$ to the internal memory of the Key Manager.
- (c) Wait for the Key Manager to complete the deployment.
- (d) Read $k2 * G$ and *key_info* (corresponding to the negotiated private key $k1 * k2 * G$) from the internal memory of the Key Manager by specifying the address.

Notes:

- NIST-P256 curve is used in the ECDH1 mode, all the ECC point multiplication is calculated on NIST-P256 curve.
- ECDH1 Deploy Mode is suitable for factory production: the manufacturer provides the factory with *init_key* and the firmware encrypted with $k1 * k2 * G$, and store $k2_info$ and $k1 * G$ in the firmware.
 - In this case, it is recommended to include a private key first-time deployment process in the firmware, to store *key_info*. In the subsequent chip power-ups, use the recovery deployment mode, instead of deploying via ECDH1 mode each time. This helps effectively defend against eFuse slice attacks.
 - Note that this deployment method requires a trusted factory, as the factory will have access to the plaintext *init_key* (used to be burned into eFuse). If the factory is untrusted and knows how to use *init_key* to decrypt $k2_info$ into $k2$, it may result in a potential leakage of $k1 * k2 * G$.
- Similar to AES Deploy Mode, when using ECDH1 Deploy Mode to deploy the key each time, it is essential to ensure that the values of *init_key*, $k2_info$, and $k1 * G$ are not leaked simultaneously.
 - If using *init_key* stored in EFUSE_KM_INIT_KEY eFuse block, $k2_info$ and $k1 * G$ may be transmitted in an untrusted environment. However, this requires to locally retain the *init_key* that was written to EFUSE_KM_INIT_KEY eFuse block at the first time.
 - If using *sw_init_key*, the transmission of *sw_init_key*, $k2_info$, and $k1 * G$ must occur in a trusted environment.
- When updating keys using the ECDH1 Deploy Mode, it provides higher security performance compared to the AES Deploy Mode. Since the value of $k1 * G$ is independent of $k2$ and *init_key*, you can update the negotiated key in ECDH1 mode by updating only $k2_info$ without changing $k1 * G$. Users can:
 - Write $k1 * G$ into the firmware in a trusted environment.
 - When updating the key using the ECDH1 Deploy Mode, transmit only *init_key* and $k2_info$. In this case, users can deploy the key in ECDH1 mode using *sw_init_key* even in an untrusted environment.

34.6.2.5 Private Key Recovery Mode

After deploying the private key, users can restore it using the *key_info* generated during deployment. The specific process is as follows.

1. Read the *key_info* from the external memory and store it to the fixed address in the internal memory of Key Manager.
2. Wait for the Key Manager to complete the deployment.
3. Repeat Step 1 ~ Step 2 above until all required private keys are restored.

Note that if two or more private keys with the same *key_purpose* are restored, only the last one will take effect. For more information about *key_purpose*, see Section 4.

34.6.2.6 *key_info* Export Mode

After successfully deploying the private key, users can regenerate *key_info* and obtain it with the help of this mode.

The specific process is as follows.

1. Select the successfully deployed *key_purpose* to enter the *key_info* export mode.
2. Wait for the Key Manager process to complete.
3. Read *key_info* from the internal memory of the Key Manager.

Note:

- This mode can be used to output different *key_info* for the same key.
- This mode can also only update HUK without updating the private key. The detailed operation is:
 - Update the HUK using the HUK Generation Mode.
 - Update the *key_info* through the *key_info* Export Mode.

By such way, the chip in the next startup can use the new *key_info* to restore the same private key under the new HUK.

34.7 Programming Procedures

34.7.1 HUK Generator

The HUK generation process is divided into six phases:

1. **IDLE**: In this phase, users configure the mode and static parameters.
2. **PREP**: In this phase, *HUK_STATE* is BUSY. The HUK Generator prepares for HUK generation. Users need to wait for this phase to complete. The HUK Generator will automatically proceed to the LOAD phase.
3. **LOAD**: In this phase, users store the HUK information into the internal memory of the HUK Generator based on the selected mode.
4. **PROC**: In this phase, *HUK_STATE* is BUSY. The HUK Generator performs the HUK generation process. Users need to wait for the phase to complete. The HUK Generator will automatically proceed to the GAIN phase.
5. **GAIN**: In this phase, users read the required HUK information from the internal memory of the HUK Generator based on the selected mode.
6. **POST**: In this phase, *HUK_STATE* is BUSY. The HUK Generator completes the follow-up tasks for HUK generation. Users need to wait for this phase to complete. The HUK Generator will automatically return to the IDLE phase.

34.71.1 IDLE Phase

Users check [HUK_STATE](#) to confirm that the HUK Generator is in the IDLE phase. In the IDLE phase, users need to perform the following configurations.

1. **Set the Working Mode:** Configure the working mode of the HUK Generator as specified in Table [34.6-1](#).
2. **Transition to PREP Phase:** Set [HUK_START](#) to complete the IDLE phase and enter the PREP phase.

34.71.2 PREP Phase

Users need to wait for the PREP phase to complete. This can be accomplished using one of the following methods:

- **Monitor [HUK_STATE](#):** Continuously check [HUK_STATE](#) until it is no longer BUSY.
- **Use Interrupts:** Enable the interrupt [HUK_PREP_DONE_INT](#) and handle the completion in the interrupt service routine.

34.71.3 LOAD Phase

Users check [HUK_STATE](#) to confirm that the HUK Generator is in the LOAD phase. In the LOAD phase, users need to perform the following configurations.

1. **Configure Based on Mode:** Depending on the mode of the HUK Generator, configure as follows:
 - **HUK Generation Mode:** No additional configuration is required.
 - **HUK Recovery Mode:** Write the HUK recovery information ([huk_info](#), 165 words) into [HUK_INFO_MEM](#).

Note:

If multiple [huk_info](#) have been generated and stored in external memory previously, using any [huk_info](#) in the HUK Recovery Mode will restore its corresponding HUK.

2. **Transition to PROC Phase:** Set the bit [HUK_CONTINUE](#) to 1 to complete the LOAD phase and enter the PROC phase.

34.71.4 PROC Phase

Users need to wait for the PROC phase to complete. This can be done using one of the following methods.

- **Monitor [HUK_STATE](#):** Continuously check [HUK_STATE](#) until it is no longer BUSY.
- **Use Interrupts:** Enable the interrupt [HUK_PROC_DONE_INT](#) and handle the completion in the interrupt service routine.

34.71.5 GAIN Phase

Users check [HUK_STATE](#) to confirm that the HUK Generator is in the GAIN phase. In the GAIN phase, users extract the required key information based on the selected mode of the HUK Generator.

1. **Extract Key Information:**

- **HUK Generation Mode:** Read *huk_info* (165 words) from `HUK_INFO_MEM`.
 - **HUK Recovery Mode:** Update *huk_info* (165 words), depending on `HUK_UPDATE_REQ`:
 - 0: Do not need to update *huk_info*, or read any information.
 - 1: Must update *huk_info* by reading a new value (165 words) from `HUK_INFO_MEM`.
2. **Transition to POST Phase:** Set the bit `HUK_CONTINUE` to complete the GAIN phase and enter the POST phase.

34.7.1.6 POST Phase

Users need to wait for the POST phase to complete. This can be done using one of the following methods.

- **Monitor `HUK_STATE`:** Continuously check `HUK_STATE` until it is no longer BUSY.
- **Use Interrupts:** Enable the interrupt `HUK_POST_DONE_INT` and handle the completion in the interrupt service routine.

34.7.2 Key Manager

The key deployment process of the Key Manager is divided into six phases:

1. **IDLE:** In this phase, users configure the mode, key usage, and static parameters.
2. **PREP:** In this phase, `KEYMNG_STATE` is BUSY. The Key Manager performs the necessary preparations for key deployment. Users need to wait for this phase to complete. The Key Manager will automatically proceed to the LOAD phase.
3. **LOAD:** In this phase, users store the key information into the internal memory of the Key Manager based on the selected mode.
4. **PROC:** In this phase, `KEYMNG_STATE` is BUSY. The Key Manager performs the key deployment process. Users need to wait for this phase to complete. The Key Manager will automatically proceed to the GAIN phase.
5. **GAIN:** In this phase, users read the required key information from the internal memory of the Key Manager based on the selected mode.
6. **POST:** In this phase, `KEYMNG_STATE` is BUSY. The Key Manager completes the follow-up tasks for key deployment. Users need to wait for this phase to complete. The Key Manager will automatically return to the IDLE phase.

34.7.2.1 IDLE Phase

Users check `KEYMNG_STATE` to confirm that the Key Manager is in the IDLE phase. In the IDLE phase, users need to perform the following configurations.

1. **Complete HUK Generation Process:**
 - (a) Select one of the following two processes, depending on whether the HUK has been generated or not:
 - **If HUK has not been generated:** Follow the HUK generation process described in Section 34.7.1.

- **If HUK has already been generated:** Skip this HUK generation process.
- (b) **Verify HUK Validity:** Check [KEYMNG_HUK_VALID](#):
- 0: the HUK is invalid. Return to Step [1a](#) until its value is 1.
 - 1: the HUK is valid.
2. **Initialize TRNG correctly:** The user needs to make sure the TRNG is working correctly.
3. **Configure Static Parameters:** Set the following parameters, including eFuse bits, registers and their corresponding locks:
- **EFUSE_KM_INIT_KEY eFuse block:** 256-bit initial key used by the Key Manager in AES Deploy Mode and ECDH1 Deploy Mode.
 - **EFUSE_KM_DEPLOY_ONLY_ONCE:** Controls whether the corresponding private key can only be deployed once after a power-on.
 - **Bit[0]:** Configures whether ECDSA Digital Signature Peripheral (ECDSA_DS) key can be deployed only once.
 - * 0: ECDSA_DS key can be deployed more than once.
 - * 1: ECDSA_DS key can be deployed only once.
 - **Bit[1]:** Configures whether XTS key can be deployed only once.
 - * 0: XTS key can be deployed more than once.
 - * 1: XTS key can be deployed only once.
 - **EFUSE_FORCE_USE_KEY_MANAGER_KEY:** Controls whether the corresponding private key must come from the Key Manager.
 - **Bit[0]:** Configures whether ECDSA_DS key must come from the Key Manager.
 - * 0: ECDSA_DS key does not have to come from the Key Manager.
 - * 1: ECDSA_DS key must come from the Key Manager.
 - **Bit[1]:** Configures whether flash encryption key must come from the Key Manager.
 - * 0: Flash key does not have to come from the Key Manager.
 - * 1: Flash key must come from the Key Manager.
 - **Bit[2]:** Configures whether HMAC key must come from the Key Manager.
 - * 0: HMAC key does not have to come from the Key Manager.
 - * 1: HMAC key must come from the Key Manager.
 - **Bit[3]:** Configures whether RSA Digital Signature Peripheral (RSA_DS) key must come from the Key Manager.
 - * 0: RSA_DS key does not have to come from the Key Manager.
 - * 1: RSA_DS key must come from the Key Manager.
 - **Bit[4]:** Configures whether PSRAM key must come from the Key Manager.
 - * 0: PSRAM key does not have to come from the Key Manager.

- * 1: PSRAM key must come from the Key Manager.
- **KEYMNG_USE_EFUSE_KEY:** Configures whether the key bit comes from eFuse block, valid only when the corresponding bit in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
 - **Bit[0]:** Configures whether ECDSA_DS key comes from eFuse block.
 - * 0: No effect.
 - * 1: ECDSA_DS key comes from eFuse block, valid only when bit[0] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
 - **Bit[1]:** Configures whether flash encryption key comes from eFuse block.
 - * 0: No effect.
 - * 1: Flash key comes from eFuse block, valid only when bit[1] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
 - **Bit[2]:** Configures whether HMAC key comes from eFuse block.
 - * 0: No effect.
 - * 1: Flash key comes from eFuse block, valid only when bit[2] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
 - **Bit[3]:** Configures whether RSA Digital Signature Peripheral (RSA_DS) key comes from eFuse block.
 - * 0: No effect.
 - * 1: RSA_DS key comes from eFuse block, valid only when bit[3] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
 - **Bit[4]:** Configures whether PSRAM key comes from eFuse block.
 - * 0: No effect.
 - * 1: PSRAM key comes from eFuse block, valid only when bit[4] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
- **EFUSE_FORCE_DISABLE_SW_INIT_KEY:** Controls whether the use of software-source *init_key* (*sw_init_key*) is permanently disabled.
 - 0: Not disabled.
 - 1: Permanently disabled.
- **KEYMNG_USE_SW_INIT_KEY:** Configures whether or not to use the software configured *sw_init_key* instead of the eFuse block whose purpose is EFUSE_KM_INIT_KEY, valid only when EFUSE_FORCE_DISABLE_SW_INIT_KEY is 0.
 - 0: Use the eFuse block whose purpose is EFUSE_KM_INIT_KEY.
 - 1: Use the software configured *sw_init_key*.
- **EFUSE_KM_RND_SWITCH_CYCLE:** Configures the interval for the Key Manager to switch random numbers. It is recommended to set this switch cycle to the number of Key Manager clock cycles corresponding to the TRNG module clock cycles.
 - 0: Switch cycle is controlled by [KEYMNG_RND_SWITCH_CYCLE](#).

- 1: 8 cycles of Key Manager clock.
 - 2: 16 cycles of Key Manager clock.
 - 3: 32 cycles of Key Manager clock.
 - **KEYMNG_RND_SWITCH_CYCLE:** Configures the interval for the Key Manager to switch random numbers, valid only when EFUSE_KM_RND_SWITCH_CYCLE is 0. It is recommended to set this switch cycle to the number of Key Manager clock cycles corresponding to the TRNG module clock cycles.
4. **Configure Working Mode:** Configure [KEYMNG_KGEN_MODE](#) to complete the working mode configuration of the Key Manager as described in Section [34.6.2](#).
- 0: Random Deploy Mode
 - 1: AES Deploy Mode
 - 2: ECDH0 Deploy Mode
 - 3: ECDH1 Deploy Mode
 - 4: Private Key Recovery mode
 - 5: [key_info](#) Export Mode
 - Others: Not valid
5. **Configure Target Key:** Configure [KEYMNG_KEY_PURPOSE](#) to the target *key_purpose*:
- 1: ecdsa_key_192
 - 2: ecdsa_key_256
 - 3: flash_256_1_key
 - 4: flash_256_2_key
 - 5: flash_128_key
 - 6: hmac_key
 - 7: ds_key
 - 8: psram_256_1_key
 - 9: psram_256_2_key
 - 10: psram_128_key
 - 11: ecdsa_key_384_l
 - 12: ecdsa_key_384_h
 - Other values: Not valid
6. **Transition to PREP Phase:** Configure [KEYMNG_START](#) to complete the IDLE phase and enter the PREP phase.

34.7.2.2 PREP Phase

Users need to wait for the PREP phase to complete. This can be done using one of the following methods:

- **Monitor [KEYMNG_STATE](#):** Continuously check [KEYMNG_STATE](#) until it is no longer BUSY.
- **Use Interrupt:** Enable the interrupt [KEYMNG_PREP_DONE_INT](#) and handle the completion in the interrupt service routine.

34.7.2.3 LOAD Phase

Users check [KEYMNG_STATE](#) to confirm that the Key Manager is in the LOAD phase. In the LOAD phase, users need to perform the following configurations.

1. **Configure Based on Mode:** Depending on the mode of the Key Manager, configure as follows:

- **Random Deploy Mode:** No additional configuration is required.
- **AES Deploy Mode:**
 - (a) If using software to configure *init_key*: write *sw_init_key* (256 bits) into [KEYMNG_SW_INIT_KEY_MEM](#).
 - (b) Write *k2_info* (512 bits) into [KEYMNG_ASSIST_INFO_MEM](#).
 - (c) Write *k1_encrypted* (256 bits) into [KEYMNG_PUBLIC_INFO_MEM](#).
- **ECDH0 Deploy Mode:**
 - (a) Write $k1 * G$ (512 bits) into [KEYMNG_PUBLIC_INFO_MEM](#).
- **ECDH1 Deploy Mode:**
 - (a) If the software configured *init_key* is used: write *sw_init_key* (256 bits) into [KEYMNG_SW_INIT_KEY_MEM](#).
 - (b) Write *k2_info* (512 bits) into [KEYMNG_ASSIST_INFO_MEM](#).
 - (c) Write $k1 * G$ (512 bits) into [KEYMNG_PUBLIC_INFO_MEM](#).
- **Private Key Recovery Mode:**
 - (a) Write *key_info* (512 bits) into [KEYMNG_ASSIST_INFO_MEM](#).
- ***key_info* Export Mode:** No additional configuration is required.

2. **Transition to PROC Phase:** Configure [KEYMNG_CONTINUE](#) to complete the LOAD phase and enter the PROC phase.

34.7.2.4 PROC Phase

Users need to wait for the PROC phase to complete. This can be done using one of the following methods.

- **Monitor [KEYMNG_STATE](#):** Continuously check [KEYMNG_STATE](#) until it is no longer BUSY.
- **Use Interrupt:** Enable the interrupt [KEYMNG_PROC_DONE_INT](#) and handle the completion in the interrupt service routine.

34.7.2.5 GAIN Phase

Users check [KEYMNG_STATE](#) to confirm that the Key Manager is in the GAIN phase. In the GAIN phase, users extract the required key information based on the selected mode of the Key Manager.

1. **Extract Key Information:**

- **Random Deploy Mode:**

(a) Read *key_info* (512 bits) from [KEYMNG_PUBLIC_INFO_MEM](#).

- **AES Deploy Mode:**

(a) Read *key_info* (512 bits) from [KEYMNG_PUBLIC_INFO_MEM](#).

- **ECDHO Deploy Mode:**

(a) Read *key_info* (512 bits) from [KEYMNG_PUBLIC_INFO_MEM](#).

(b) Read $k_2 * G$ (512 bits) from [KEYMNG_ASSIST_INFO_MEM](#).

- **ECDH1 Deploy Mode:**

(a) Read *key_info* (512 bits) from [KEYMNG_PUBLIC_INFO_MEM](#).

- **Private Key Recovery Mode:** no need to read any information.

- *key_info* **Export Mode:**

(a) Read *key_info* (512 bits) from [KEYMNG_PUBLIC_INFO_MEM](#).

2. **Transition to POST Phase:** Configure [KEYMNG_CONTINUE](#) to complete the GAIN phase and enter the POST phase.

34.7.2.6 POST Phase

Users need to wait for the POST phase to complete. This can be done using one of the following methods.

- **Monitor [KEYMNG_STATE](#):** Continuously check [KEYMNG_STATE](#) until it is no longer BUSY.
- **Use Interrupt:** Enable the interrupt [KEYMNG_POST_DONE_INT](#) and handle the completion in the interrupt service routine.

34.8 Programming Examples

Based on the previous sections, we can outline a comprehensive chip deployment process. This section explains the operation procedures for the HUK Generator and Key Manager during two typical chip startup scenarios.

Note:

In this section, all configuration processes are described in a simplified manner. For detailed instructions, please refer to Section [34.7](#).

34.8.1 Software Process in Chip Private Key Deploy Phase

The following steps outline the process for booting up the chip for the first time and deploying the private key for encrypting and decrypting external memory.

1. **Enter Download Mode:** The chip enters the download mode.
2. **Configure HUK Generator:** Since it is the first time to start the chip, configure the HUK Generator to enter the HUK Generation Mode.
 - (a) Configure the IDLE phase and set the working mode as HUK Generation Mode as specified in Table 34.6-1, then configure [HUK_START](#).
 - (b) After entering the LOAD phase, configure [HUK_CONTINUE](#).
 - (c) After entering GAIN phase, read out [huk_info](#), then configure [HUK_CONTINUE](#).
 - (d) Wait for the HUK Generator to return to the IDLE phase.
3. **Set HUK Generation State:** Write EFUSE_KM_HUK_GEN_STATE to make the total number of bits set to 1 is odd, so that the HUK Generator will work only in HUK Recovery Mode during subsequent startups.
4. **Configure Key Deployment:** Take the key deployment in ECDHO Deploy Mode as an example.
 - (a) Configure the IDLE phase, set [KEYMNG_KEY_PURPOSE](#) as XTS, and configure [KEYMNG_START](#).
 - (b) After entering the LOAD phase, write the locally generated $k1 * G$ to the Key Manager, then configure [KEYMNG_CONTINUE](#).
 - (c) After entering the GAIN phase, read out [key_info](#) and $k2 * G$, then configure [KEYMNG_CONTINUE](#).
 - (d) Wait for the Key Manager to return to the IDLE phase.
5. **Upload Key:** Upload $k2 * G$ to the user through the communication interface.
6. **Generate and Encrypt Data:**
 - Generate the negotiated private key $k1 * k2 * G$ based on the obtained $k2 * G$ and the local $k1$.
 - Encrypt data locally and write the ciphertext to flash.

34.8.2 Software Process in Chip Normal Startup Phase

Once the chip has been deployed, the complete process for a normal startup is as follows.

1. **Start in SPI Boot Mode:**

The chip begins in SPI boot mode and first enters the unencrypted program segment.

2. **Configure HUK Recovery Mode:**

Since the HUK has been generated, configure the HUK Recovery Mode as follows:

- (a) Configure the IDLE phase and set the working mode as HUK Recovery Mode as specified in Table 34.6-1, then configure [HUK_START](#).
- (b) After entering the LOAD phase, write [huk_info](#) into the HUK Generator, then configure [HUK_CONTINUE](#).
- (c) After entering GAIN phase, configure [HUK_CONTINUE](#).

(d) Wait for the HUK Generator to return to the IDLE phase.

3. Configure Key Manager for Private Key Recovery Mode:

(a) Configure the IDLE phase and set `KEYMNG_KEY_PURPOSE` as `flash_128_key`, then configure `KEYMNG_START`.

(b) After entering the LOAD phase, write `key_info` into the Key Manager, then configure `KEYMNG_CONTINUE`.

(c) After entering the GAIN phase, configure `KEYMNG_CONTINUE`.

(d) Wait for the Key Manager to return to the IDLE phase.

4. Enter Encrypted Program Segment:

At this point, the encryption/decryption private key in the external memory has taken effect, and the system enters the encrypted program segment.

34.9 Interrupts

ESP32-P4's Key Manager and HUK Generator can generate the following interrupt signals that will be sent to the [Interrupt Matrix](#).

- `KEYMNG_INTR`
- `HUK_INTR`

There are several internal interrupt sources from Key Manager and HUK Generator that can generate the above interrupt signals. The interrupt sources from Key Manager and HUK Generator are listed with their trigger conditions and the resulted interrupt signals in Table 34.9-1.

Table 34.9-1. Key Manager and HUK Generator's Internal Interrupt Sources

Internal Interrupt Source	Trigger Condition	Interrupt Signal
<code>KEYMNG_PREP_DONE_INT</code>	Completion of Key Manager's PREP phase	<code>KEYMNG_INTR</code>
<code>KEYMNG_PROC_DONE_INT</code>	Completion of Key Manager's PROC phase	<code>KEYMNG_INTR</code>
<code>KEYMNG_POST_DONE_INT</code>	Completion of Key Manager's POST phase	<code>KEYMNG_INTR</code>
<code>HUK_PREP_DONE_INT</code>	Completion of HUK Generator's PREP phase	<code>HUK_INTR</code>
<code>HUK_PROC_DONE_INT</code>	Completion of HUK Generator's PROC phase	<code>HUK_INTR</code>
<code>HUK_POST_DONE_INT</code>	Completion of HUK Generator's POST phase	<code>HUK_INTR</code>

Note:

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter 12 [Interrupt Matrix](#) > Section 12.2 [Interrupt Terminology in ESP32-P4](#).

34.10 Memory Blocks

The addresses in this section are relative to Key Manager or HUK Generator base address provided in Table 7.3-2 in Chapter 7 [System and Memory](#).

Table 34.10-1. Key Manager Memory Blocks

Name	Description	Size (byte)	Starting Address	Ending Address	Access
KEYMNG_ASSIST_INFO_MEM	Stores assistant information	64	0x0100	0x013F	Varies
KEYMNG_PUBLIC_INFO_MEM	Stores public information	64	0x0140	0x017F	Varies
KEYMNG_SW_INIT_KEY_MEM	Stores <i>sw_init_key</i>	32	0x0180	0x019F	Varies

Table 34.10-2. HUK Generator Memory Blocks

Name	Description	Size (byte)	Starting Address	Ending Address	Access
HUK_INFO_MEM	Stores <i>huk_info</i>	660	0x0100	0x0393	Varies

34.11 Register Summary

34.11.1 HUK Register Summary

The addresses in this section are relative to HUK Generator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Clock Gate Register			
HUK_CLK_REG	Clock gate control register	0x0004	R/W
Interrupt Registers			
HUK_INT_RAW_REG	Interrupt raw register (valid in level)	0x0008	RO/WTC/SS
HUK_INT_ST_REG	Interrupt status register	0x000C	RO
HUK_INT_ENA_REG	Interrupt enable register	0x0010	R/W
HUK_INT_CLR_REG	Interrupt clear register	0x0014	WT
Configuration Register			
HUK_CONF_REG	Configuration register	0x0020	R/W
Control Register			
HUK_START_REG	Control register	0x0024	WT
State Register			
HUK_STATE_REG	HUK Generator state register	0x0028	RO
Result Register			
HUK_STATUS_REG	HUK status register	0x0034	RO
Version Register			
HUK_DATE_REG	Version control register	0x00FC	R/W

34.11.2 Key Manager Register Summary

The addresses in this section are relative to Key Manager base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Clock Gate Register			
KEYMNG_CLK_REG	Clock gate control register	0x0004	R/W
Interrupt Registers			
KEYMNG_INT_RAW_REG	Interrupt raw register (valid in level)	0x0008	RO/WTC/SS
KEYMNG_INT_ST_REG	Interrupt status register	0x000C	RO
KEYMNG_INT_ENA_REG	Interrupt enable register	0x0010	R/W
KEYMNG_INT_CLR_REG	Interrupt clear register	0x0014	WT
Static Configuration Registers			
KEYMNG_STATIC_REG	Static configuration register	0x0018	R/W
KEYMNG_LOCK_REG	Static configuration lock register	0x001C	R/W1

Name	Description	Address	Access
Configuration register			
KEYMNG_CONF_REG	Configuration register	0x0020	R/W
Control Register			
KEYMNG_START_REG	Control register	0x0024	WT
State Register			
KEYMNG_STATE_REG	State register	0x0028	RO
Result Registers			
KEYMNG_RESULT_REG	Operation result register	0x002C	RO/SS
KEYMNG_KEY_VLD_REG	Key status register	0x0030	RO
KEYMNG_HUK_VLD_REG	HUK status register	0x0034	RO
Version Register			
KEYMNG_DATE_REG	Version control register	0x00FC	R/W

34.12 Registers

34.12.1 HUK Registers

The addresses in this section are relative to HUK base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section IX .

Register 34.1. HUK_CLK_REG (0x0004)

(reserved)																															HUK_MEM_CG_FORCE_ON HUK_CLK_EN					
31																															2	1	0			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	Reset

HUK_CLK_EN Configures whether or not to force on register clock gate.

0: Not force on

1: Force on

(R/W)

HUK_MEM_CG_FORCE_ON Configures whether or not to force on memory clock gate.

0: Not force on

1: Force on

(R/W)

Register 34.2. HUK_INT_RAW_REG (0x0008)

(reserved)																												HUK_POST_DONE_INT_RAW HUK_PROC_DONE_INT_RAW HUK_PREP_DONE_INT_RAW																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													
31																												3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0</

HUK_PREP_DONE_INT_RAW The raw interrupt status of [HUK_PREP_DONE_INT](#) interrupt.

(RO/WTC/SS)

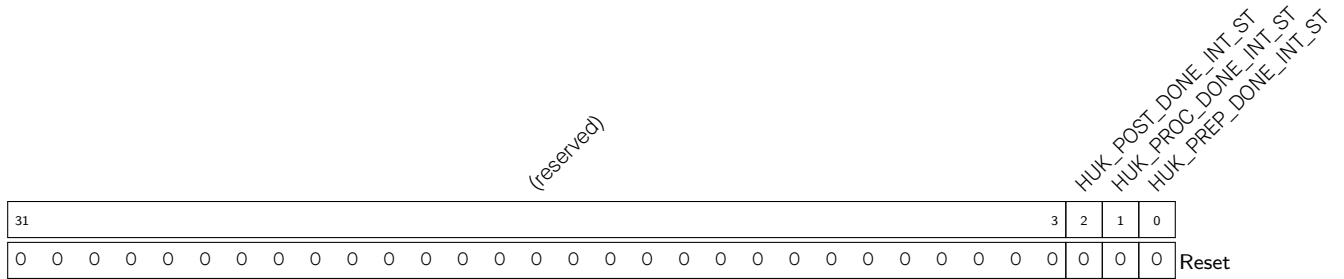
HUK_PROC_DONE_INT_RAW The raw interrupt status of [HUK_PROC_DONE_INT](#) interrupt.

(RO/WTC/SS)

HUK_POST_DONE_INT_RAW The raw interrupt status of [HUK_POST_DONE_INT](#) interrupt.

(RO/WTC/SS)

Register 34.3. HUK_INT_ST_REG (0x000C)

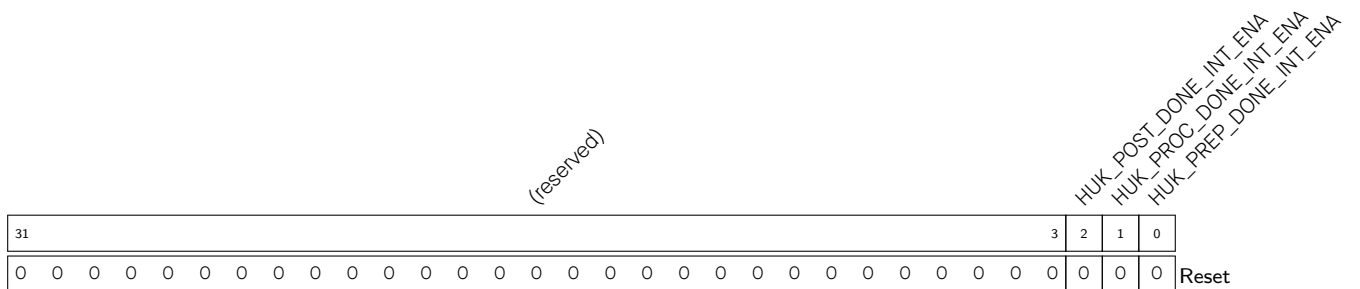


HUK_PREP_DONE_INT_ST The masked interrupt status of [HUK_PREP_DONE_INT](#) interrupt.
(RO)

HUK_PROC_DONE_INT_ST The masked interrupt status of [HUK_PROC_DONE_INT](#) interrupt.
(RO)

HUK_POST_DONE_INT_ST The masked interrupt status of [HUK_POST_DONE_INT](#) interrupt.
(RO)

Register 34.4. HUK_INT_ENA_REG (0x0010)



HUK_PREP_DONE_INT_ENA Write 1 to enable the [HUK_PREP_DONE_INT](#) interrupt.
(R/W)

HUK_PROC_DONE_INT_ENA Write 1 to enable the [HUK_PROC_DONE_INT](#) interrupt.
(R/W)

HUK_POST_DONE_INT_ENA Write 1 to enable the [HUK_POST_DONE_INT](#) interrupt.
(R/W)

Register 34.5. HUK_INT_CLR_REG (0x0014)

(reserved)																															HUK_POST_DONE_INT_CLR HUK_PROC_DONE_INT_CLR HUK_PREP_DONE_INT_CLR																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
31																															3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

HUK_PREP_DONE_INT_CLR Write 1 to clear the [HUK_PREP_DONE_INT](#) interrupt.
(WT)

HUK_PROC_DONE_INT_CLR Write 1 to clear the [HUK_PROC_DONE_INT](#) interrupt.
(WT)

HUK_POST_DONE_INT_CLR Write 1 to clear the [HUK_POST_DONE_INT](#) interrupt.
(WT)

Register 34.6. HUK_CONF_REG (0x0020)

(reserved)																															HUK_MODE		
31																															1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

HUK_MODE Configures HUK mode.
0: HUK Recovery Mode
1: HUK Generation Mode
(R/W)

Register 34.7. HUK_START_REG (0x0024)

(reserved)																															HUK_CONTINUE		HUK_START		
31																															2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

HUK_START Write 1 to start HUK Generator at IDLE phase.
(WT)

HUK_CONTINUE Write 1 to continue HUK Generator operation at LOAD/GAIN phase.
(WT)

Register 34.8. HUK_STATE_REG (0x0028)

(reserved)																												HUK_STATE		
31																												2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

HUK_STATE Represents the state of HUK Generator.

- 0: IDLE
 - 1: LOAD
 - 2: GAIN
 - 3: BUSY
- (RO)

Register 34.9. HUK_STATUS_REG (0x0034)

(reserved)																												HUK_UPDATE_REQ			HUK_RISK_LEVEL			HUK_STATUS																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
31																												6	5	4	2		1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

HUK_STATUS Represents the HUK generation status.

- 0: HUK is not generated.
 - 1: HUK is generated and valid.
 - 2: HUK is generated but invalid.
 - 3: Reserved.
- (RO)

HUK_RISK_LEVEL Represents the risk level of HUK.

- 0~6: The higher the risk level is, the more error bits there are in the PUF SRAM.
 - 7: Error Level, HUK is invalid.
- (RO)

HUK_UPDATE_REQ Represents the update request of *huk_info*.

- 0: User can update *huk_info* according to the risk level.
 - 1: The *huk_info* is expired, and user need to update it.
- (RO)

Register 34.10. HUK_DATE_REG (0x00FC)

(reserved)				HUK_DATE																									
31	28	27																										0	
0	0	0	0	0x2404070																									Reset

HUK_DATE Version control register.
(R/W)

34.12.2 Key Manager Registers

The addresses in this section are relative to Key Manager base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section IX .

Register 34.11. KEYMNG_CLK_REG (0x0004)

(reserved)																											KEYMNG_MEM_CG_FORCE_ON KEYMNG_REG_CG_FORCE_ON						
31																										2	1	0					
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	Reset

KEYMNG_REG_CG_FORCE_ON Configures whether or not to force on register clock gate.
0: Not force on
1: Force on
(R/W)

KEYMNG_MEM_CG_FORCE_ON Configures whether or not to force on memory clock gate.
0: Not force on
1: Force on
(R/W)

Register 34.12. KEYMNG_INT_RAW_REG (0x0008)

[illegible]

KEYMNG_PREP_DONE_INT_RAW The raw interrupt status of [KEYMNG_PREP_DONE_INT](#) interrupt.
(RO/WTC/SS)

KEYMNG_PROC_DONE_INT_RAW The raw interrupt status of [KEYMNG_PROC_DONE_INT](#) interrupt.
(RO/WTC/SS)

KEYMNG_POST_DONE_INT_RAW The raw interrupt status of [KEYMNG_POST_DONE_INT](#) interrupt.
(RO/WTC/SS)

Register 34.13. KEYMNG_INT_ST_REG (0x000C)

Diagram of the KEYMNG_PREP_DONE_INT_ST register structure:

- Bit 31: (reserved)
- Bits 3-2: (reserved)
- Bit 1: KEYMNG_PREP_DONE_INT_ST
- Bit 0: KEYMNG_PREP_DONE_INT_ST

KEYMNG_PREP_DONE_INT_ST The masked interrupt status of [KEYMNG_PREP_DONE_INT](#) interrupt.
(RO)

KEYMNG_PROC_DONE_INT_ST The masked interrupt status of [KEYMNG_PROC_DONE_INT](#) interrupt.
(RO)

KEYMNG_POST_DONE_INT_ST The masked interrupt status of **KEYMNG_POST_DONE_INT** interrupt.
(RO)

Register 34.14. KEYMNG_INT_ENA_REG (0x0010)

(reserved)																																KEYMNG_POST_DONE_INT_ENA KEYMNG_PROC_DONE_INT_ENA KEYMNG_PREP_DONE_INT_ENA																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
31																																3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

KEYMNG_PREP_DONE_INT_ENA Write 1 to enable the [KEYMNG_PREP_DONE_INT](#) interrupt.
(R/W)

KEYMNG_PROC_DONE_INT_ENA Write 1 to enable the [KEYMNG_PROC_DONE_INT](#) interrupt.
(R/W)

KEYMNG_POST_DONE_INT_ENA Write 1 to enable the [KEYMNG_POST_DONE_INT](#) interrupt.
(R/W)

Register 34.15. KEYMNG_INT_CLR_REG (0x0014)

(reserved)																															KEYMNG_POST_DONE_INT_CLR KEYMNG_PROC_DONE_INT_CLR KEYMNG_PREP_DONE_INT_CLR																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
31																															3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

KEYMNG_PREP_DONE_INT_CLR Write 1 to clear the [KEYMNG_PREP_DONE_INT](#) interrupt.
(WT)

KEYMNG_PROC_DONE_INT_CLR Write 1 to clear the [KEYMNG_PROC_DONE_INT](#) interrupt.
(WT)

KEYMNG_POST_DONE_INT_CLR Write 1 to clear the [KEYMNG_POST_DONE_INT](#) interrupt.
(WT)

Register 34.16. KEYMNG_STATIC_REG (0x0018)

(reserved)																KEYMNG_PSRAM_KEY_LEN				KEYMNG_FLASH_KEY_LEN				KEYMNG_USE_SW_INIT_KEY				KEYMNG_RND_SWITCH_CYCLE				KEYMNG_USE_EFUSE_KEY			
31																13				12	11	10	9				5				4	0			
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0				0	0	0	15				0				Reset				

Reset

KEYMNG_USE_EFUSE_KEY Configures whether or not to use eFuse key instead of Key Manager deployed key.

- Bit[0]: configures whether to use eFuse key for ECDSA key.
0: No effect
1: Use eFuse Key, valid only when bit[0] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
- Bit[1] configures whether to use eFuse key for flash key.
0: No effect
1: Use eFuse Key, valid only when bit[1] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
- Bit[2] configures whether to use eFuse key for HMAC key.
0: No effect
1: Use eFuse Key, valid only when bit[2] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
- Bit[3] configures whether to use eFuse key for RSA_DS_KEY.
0: No effect
1: Use eFuse Key, valid only when bit[3] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.
- Bit[4] configures whether to use eFuse key for PSRAM key.
0: No effect
1: Use eFuse Key, valid only when bit[4] in EFUSE_FORCE_USE_KEY_MANAGER_KEY is 0.

(R/W)

KEYMNG_RND_SWITCH_CYCLE Configures cycles of the Key Manager to switch random numbers.

Measurement: Key Manager clock cycle.

This field is valid only when EFUSE_KM_RND_SWITCH_CYCLE is set to 0.

It is recommended to set this switch cycle to the number of Key Manager clock cycles corresponding to the TRNG module's clock cycle.

(R/W)

KEYMNG_USE_SW_INIT_KEY Configures whether or not to use *sw_init_key*, instead of EFUSE_KM_INIT_KEY, valid only when EFUSE_FORCE_DISABLE_SW_INIT_KEY is 0.

0: Use EFUSE_KM_INIT_KEY

1: Use software written *sw_init_key*

(R/W)

Continued on the next page...

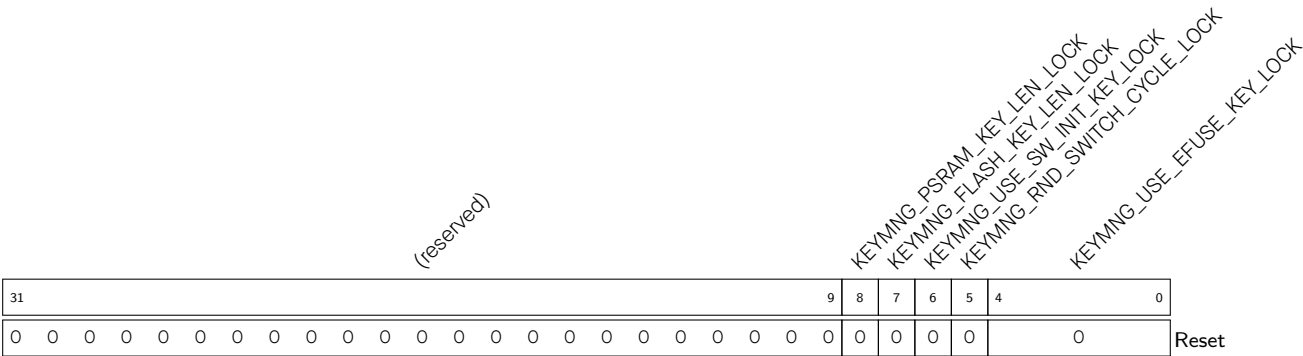
Register 34.16. KEYMNG_STATIC_REG (0x0018)

Continued from the previous page...

KEYMNG_FLASH_KEY_LEN Configures the key length for flash crypt.
0: Use xts-aes-128
1: Use xts-aes-256
(R/W)

KEYMNG_PSRAM_KEY_LEN Configures the key length for PSRAM crypt.
0: Use xts-aes-128
1: Use xts-aes-256
(R/W)

Register 34.17. KEYMNG_LOCK_REG (0x001C)



KEYMNG_USE_EFUSE_KEY_LOCK Write 1 to lock [KEYMNG_USE_EFUSE_KEY](#).
Each bit locks the corresponding bit of [KEYMNG_USE_EFUSE_KEY](#).
(R/W1)

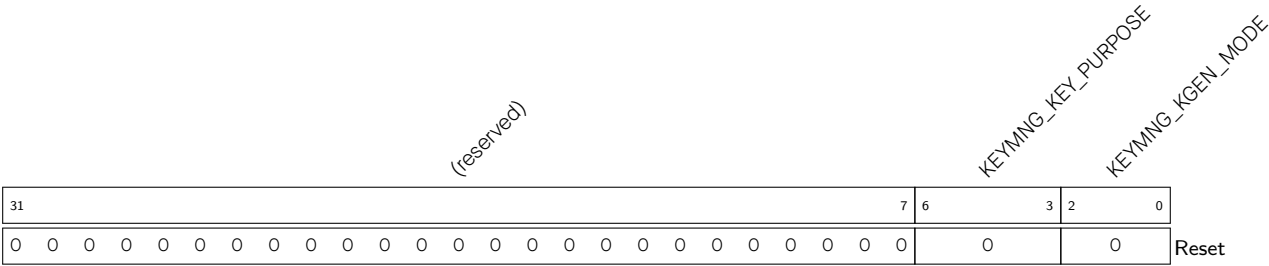
KEYMNG_RND_SWITCH_CYCLE_LOCK Write 1 to lock [KEYMNG_RND_SWITCH_CYCLE](#).
(R/W1)

KEYMNG_USE_SW_INIT_KEY_LOCK Write 1 to lock [KEYMNG_USE_SW_INIT_KEY](#).
(R/W1)

KEYMNG_FLASH_KEY_LEN_LOCK Write 1 to lock [KEYMNG_FLASH_KEY_LEN](#).
(R/W1)

KEYMNG_PSRAM_KEY_LEN_LOCK Write 1 to lock [KEYMNG_PSRAM_KEY_LEN](#).
(R/W1)

Register 34.18. KEYMNG_CONF_REG (0x0020)



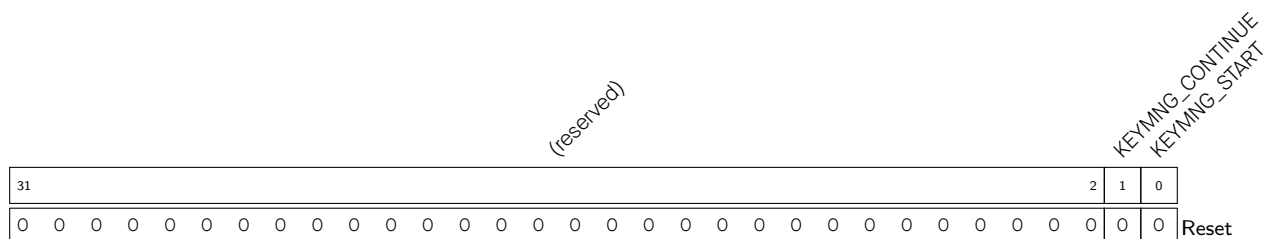
KEYMNG_KGEN_MODE Configures deployment mode.

- 0: Random Deploy Mode
 - 1: AES Deploy Mode
 - 2: ECDH0 Deploy Mode
 - 3: ECDH1 Deploy Mode
 - 4: Private Key Recovery Mode
 - 5: *key_info* Export Mode
 - 6~7: Reserved
- (R/W)

KEYMNG_KEY_PURPOSE Configures key purpose.

- 1: ecdsa_key_192
 - 2: ecdsa_key_256
 - 3: flash_256_1_key
 - 4: flash_256_2_key
 - 5: flash_128_key
 - 6: hmac_key
 - 7: ds_key
 - 8: psram_256_1_key
 - 9: psram_256_2_key
 - 10: psram_128_key
 - 11: ecdsa_key_384_l
 - 12: ecdsa_key_384_h
 - Others: Reserved
- (R/W)

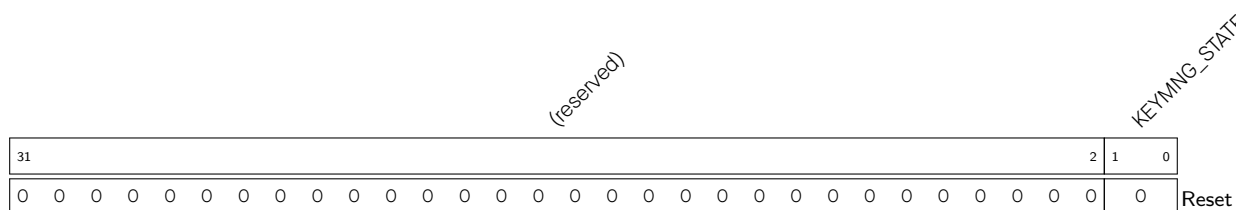
Register 34.19. KEYMNG_START_REG (0x0024)



KEYMNG_START Write 1 to start Key Manager operation at IDLE phase.
(WT)

KEYMNG_CONTINUE Write 1 to continue the operation of Key Manager at LOAD/GAIN phase.
(WT)

Register 34.20. KEYMNG_STATE_REG (0x0028)

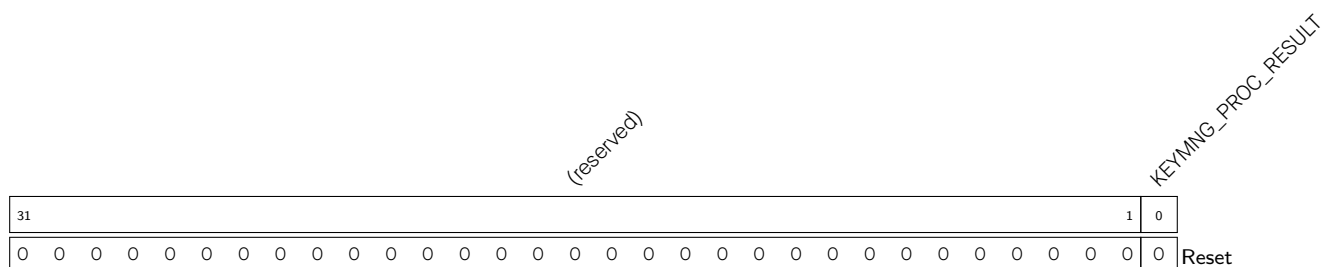


KEYMNG_STATE Represents the state of Key Manager.

- 0: IDLE
- 1: LOAD
- 2: GAIN
- 3: BUSY

(RO)

Register 34.21. KEYMNG_RESULT_REG (0x002C)



KEYMNG_PROC_RESULT Represents the procedure result of Key Manager, valid only when Key Manager procedure is done.

0: Key Manager procedure failed.

1: Key Manager procedure succeeded.

(RO/SS)

Register 34.22. KEYMNG_KEY_VLD_REG (0x0030)

(reserved)																															KEYMNG_KEY_ECDSA_384_VLD								KEYMNG_KEY_PSRAM_VLD								KEYMNG_KEY_DS_VLD								KEYMNG_KEY_HMAC_VLD								KEYMNG_KEY_FLASH_VLD								KEYMNG_KEY_ECDSA_256_VLD								KEYMNG_KEY_ECDSA_192_VLD																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
31																															7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
0																															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

KEYMNG_KEY_ECDSA_192_VLD Represents the status of key_ecdsa_192.

0: The key has not been deployed yet.

1: The key has been deployed correctly.

(RO)

KEYMNG_KEY_ECDSA_256_VLD Represents the status of key_ecdsa_256.

0: The key has not been deployed yet.

1: The key has been deployed correctly.

(RO)

KEYMNG_KEY_FLASH_VLD Represents the status of key_flash.

0: The key has not been deployed yet.

1: The key has been deployed correctly.

(RO)

KEYMNG_KEY_HMAC_VLD Represents the status of key_hmac.

0: The key has not been deployed yet.

1: The key has been deployed correctly.

(RO)

KEYMNG_KEY_DS_VLD Represents the status of key_ds.

0: The key has not been deployed yet.

1: The key has been deployed correctly.

(RO)

KEYMNG_KEY_PSRAM_VLD Represents the status of key_psram.

0: The key has not been deployed yet.

1: The key has been deployed correctly.

(RO)

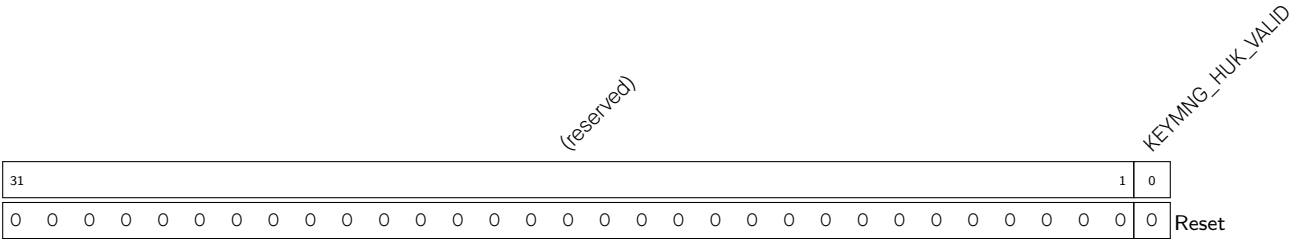
KEYMNG_KEY_ECDSA_384_VLD Represents the status of key_ecdsa_384.

0: The key has not been deployed yet.

1: The key has been deployed correctly.

(RO)

Register 34.23. KEYMNG_HUK_VLD_REG (0x0034)

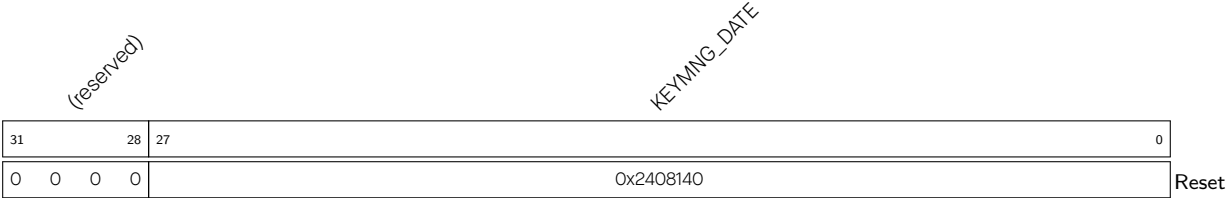


KEYMNG_HUK_VALID Represents the HUK status.

- 0: HUK is not valid.
- 1: HUK is valid.

(RO)

Register 34.24. KEYMNG_DATE_REG (0x00FC)



KEYMNG_DATE Version control register.

(R/W)